

The C library `sscanf(const char *str, const char *format, ...)` function reads formatted input from a string and stores the result in the provided variables.

Syntax

Following is the C library syntax of the `sscanf()` function

```
int sscanf(const char *str, const char *format, ...);
```

Parameters

This function accepts the following parameters

- `str`: The input string from which to read.
- `format`: A format string that specifies how to interpret the input string.
- `...`: Additional arguments pointing to variables where the extracted values will be stored.

Return Value

The function returns the number of input items successfully matched and assigned. If the input does not match the format string, the function returns EOF or the number of successfully matched and assigned items up to that point.

Format Specifiers

Below is the list of format specifier

- `%d`: Reads an integer.
- `%f`: Reads a floating-point number.
- `%s`: Reads a string.
- `%c`: Reads a character.
- `%x`: Reads a hexadecimal integer.
- `%o`: Reads an octal integer.
- `%u`: Reads an unsigned integer.

Example 1: Parsing Integers and string

The below code demonstrates parsing an integer and a string from the input.

Below is the illustration of the C library `sscanf()` function.

```
#include <stdio.h>

int main() {
    const char *input = "42 Alice";
    int number;
    char name[20];

    int result = sscanf(input, "%d %s", &number, name);

    printf("Parsed number: %d\n", number);
    printf("Parsed name: %s\n", name);
    printf("Number of items matched: %d\n", result);

    return 0;
}
```

Output

The above code produces following result

```
Parsed number: 42
Parsed name: Alice
Number of items matched: 2
```

Example 2: Parsing a Hexadecimal and a Character

This example parses a hexadecimal number and a character from the input string.

```
#include <stdio.h>
```

```
int main() {
```

```
const char *input = "0xA5 Z";
int hexValue;
char character;

int result = sscanf(input, "%x %c", &hexValue, &character);

printf("Parsed hex value: %x\n", hexValue);
printf("Parsed character: %c\n", character);
printf("Number of items matched: %d\n", result);

return 0;
}
```

Output

After execution of above code, we get the following result

```
Parsed hex value: a5
Parsed character: Z
Number of items matched: 2
```